

Keyboard Optimisation

Akash Somasundaram ee24b084

September 2025

1 Introduction

The program optimizes the QWERTY keyboard layout to minimize the total distance traveled while typing out a certain text. The optimization is done via simulated annealing, a probabilistic optimization algorithm that searches for a global minimum by gradually reducing randomness in exploring solutions. This report explains my approach to program the solution and briefly explains changes in the result when varying parameters.

2 Functions implemented

2.1 `qwerty_coordinates`

Returns the qwerty grid coordinates, i.e, a layout containing characters along with their cartesian coordinates.

2.2 `path_length_cost`

Computes the total Euclidean distance (referred to as cost) to type out the text with the given layout.

2.3 `swap_chars`

Returns a new layout after having swapped any 2 keys at random from a given layout.

2.4 `simulated_annealing`

Creates a new layout with each iteration, if the cost for this layout is better than the current layout, it accepts this new layout. However, if the cost is worse, it accepts this new layout with a certain probability. This probability is dependent on how much worse the new cost is compared to the current cost and well as the temperature. The **temperature** is a control parameter that governs the probability of accepting worse solutions, gradually decreasing to refine the search. As the iterations run, the function stores the current cost and best cost

(computed so far) in 2 separate lists. It finally returns the best cost, best layout and the two lists.

3 Final Results

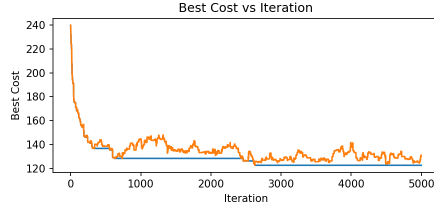


Figure 1: Cost trace

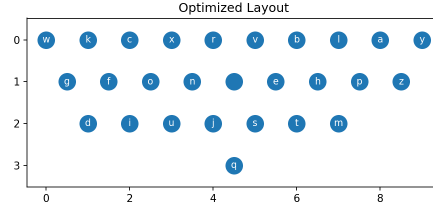


Figure 2: Optimized layout

The above shown cost trace depicts how the best cost and current cost varies as the iterations run. The algorithm accepts worse solutions to escape local minimas.

4 Varying Parameters

When the **temperature is higher**, the algorithm accepts worse solutions more often, allowing broader exploration of layouts but leading to slower convergence. As the temperature decreases, exploration reduces and the search focuses on refining the best layouts. With **more iterations**, the algorithm has more opportunities to escape local minima and converge to a better solution, while too few iterations may stop the search prematurely and yield suboptimal layouts.

Ideally, a higher temperature run over more iterations with a slow cooling schedule gives the best results. However, beyond certain limits the run-time becomes excessively large and impractical. Thus, an effective balance between temperature, decay rate, and iterations is necessary to obtain good results within a reasonable run-time.

5 How to Vary Parameters

You can run my code with different parameters by changing the values given under the `main()` function. The annealing parameters (`iters`, `t0`, `alpha`, `epoch`) used in the code control the number of iterations, initial temperature, cooling rate, and update interval.

```
params = SParams(iters=5000, t0=1.0, alpha=0.99, epoch=1)
```

Change the values in this line of the code (line 227), under the main function.